

Procedure Step Recognition on Egocentric Assembly Video:

From Procedure Step Recognition Research to an Operational Assembly Copilot

Ego-centric PSR Project
IndustReal, Engine-Model Dataset & Assembly Copilot

Abstract

We study *procedure step recognition* (PSR) on egocentric assembly video: given a first-person video of a construction task, output a dense timeline of part labels, time intervals, and correctness status. We frame PSR as temporal action segmentation with a two-stage recipe: a *frozen* video backbone produces per-clip features and only a lightweight segmentation head is trained. Starting from a VideoMAEv2-Huge + ASFormer baseline, we present a controlled progression in which *every architecture change yields a measurable improvement*: procedure-aware Viterbi decoding, a stronger SSv2-pretrained giant backbone, a diffusion segmentation head (DiffAct), and a two-encoder feature fusion (VideoMAEv2 + InternVideo2). The final model, **v4 = Fusion + DiffAct**, reaches **F1@50 = 79.50** on the IndustReal test split in the updated project results. We further build a causal real-time variant, confirm the recipe generalizes to a second dataset (MECCANO), and analyze the *correctness* sub-problem, which we show to be data-limited rather than method-limited. Finally, we add an inference-time **SAM3 enrichment layer** that augments the timeline with per-step object masks and a component inventory. We report that promptable foundation models (LocateAnything, SAM3) reliably *localize* parts but do not *judge* correctness, and we position SAM3 as an output-enrichment component rather than a correctness classifier. We then translate the research findings into a deployment-oriented Assembly Copilot: we record and label an in-house turbofan corpus, replace the non-causal diffusion head with a causal temporal convolutional network, train against order and cold-start stress cases, and validate both replay and live inference. On the collected engine-model dataset, the offline v4 model reaches Acc/Edit/F1@50 = 96.21/97.33/97.51, while the final online V1 reaches 94.12/84.45/89.50. A later robustness gate for the deployed checkpoint reaches 93.06% on normal held-out recordings and 90.57% on scrambled-order stress recordings, with a measured live announcement delay of 2.84s.

1 Introduction

Egocentric video of manual assembly is a natural testbed for fine-grained procedure understanding: the camera sees the worker’s hands and the evolving object state, and errors are induced deliberately. We target **IndustReal** [1], a dataset of 84 recordings of a toy construction (STEMFIE) split 36/16/32 into train/val/test, annotated with procedure-step completions including *incorrectly installed* and *removed* events. Our goal is a system that emits, for a full recording, a dense per-step timeline with a correctness verdict per step.

We adopt the classic two-stage temporal action segmentation (TAS) recipe [3, 4]: a large, *frozen* self-supervised video backbone provides per-clip features, and a comparatively small head is trained on top. This keeps training cheap and lets us treat the backbone and head as independent design axes. The contributions of this report are:

- A **controlled architecture progression** in which each change—decoding, backbone, head, and feature fusion—improves segmentation quality, culminating in **v4 (Fusion + DiffAct)**, **F1@50 = 79.50** (Table 1).
- A **causal real-time** variant and a **cross-dataset** confirmation on MECCANO [2].
- An analysis showing the **correctness** problem is *data-limited*, not method-limited.
- A **SAM3 enrichment layer** [12] that adds per-step masks and a component inventory at inference, together with a negative but useful result: promptable models locate but do not judge correctness.

2 Background and Related Work

Temporal action segmentation. MS-TCN [3] popularized multi-stage temporal convolutions for per-frame action labeling; ASFormer [4] introduced a transformer encoder–decoder for the same task. DiffAct [5] recasts segmentation as an iterative denoising (diffusion) process and yields sharp boundaries. **Video backbones.** VideoMAEv2 [6] is a masked-autoencoder video transformer; we use its Huge (K710) and giant (SSv2) variants. InternVideo2 [7] is a strong appearance/semantic video encoder that we fuse with VideoMAEv2. **Streaming.** MiniROAD [8] and TeSTra [9] inform our causal head design. **Promptable foundation models.** SAM [10] and SAM2 [11] established promptable segmentation; SAM3 [12] adds open-vocabulary *concept* segmentation and video tracking. LocateAnything [13] (built on Qwen2.5 [14]) is a grounding VLM that returns boxes/points. We evaluate the latter two for a correctness/enrichment role.

3 Method: the two-stage pipeline

For a recording we sample 16-frame clips at a fixed stride and pass each clip through the frozen backbone to obtain a feature $x_t \in \mathbb{R}^D$; the sequence $\{x_t\}$ is fed to the trained head, which predicts a per-clip **step** label (10 parts + background = 11 classes). Consecutive equal labels are merged into segments, giving the “which part / when” timeline. A parallel **type** head predicts correct / incorrect / remove and is pooled per segment. Metrics follow the MS-TCN protocol [3]: frame accuracy, segmental edit score, and F1 at IoU thresholds {10, 25, 50}%.

4 Architecture progression

We now walk through the design changes; each is evaluated on the IndustReal test split (Table 1).

v1 — VideoMAEv2-Huge + ASFormer (baseline). A distilled Huge (K710, 1280-d) backbone with an ASFormer head establishes the baseline at F1@50 = 56.3.

+ **Viterbi decoding.** Replacing raw arg-max with a procedure-aware Viterbi smoothing over the head’s log-probabilities removes spurious short segments and lifts the boundary metrics substantially: F1@50 62.2 (+5.9), edit 68.8 → 74.2.

v2 — SSv2-giant features. Swapping the backbone for a VideoMAEv2 *giant* model fine-tuned on Something-Something-v2 (1408-d), with no center crop and a finer stride, improves temporal grounding: F1@50 66.5 (+4.3).

v2 — DiffAct head. Replacing ASFormer with the diffusion head DiffAct [5] on the same SSv2 features yields the sharpest boundaries so far: F1@50 68.9, edit 79.6.

Fusion — adding InternVideo2. Motivated by the correctness sub-problem, we fuse a motion-centric encoder (giant SSv2, 1408-d) with an appearance/semantic encoder (InternVideo2-B14, 768-d) by ℓ_2 -normalizing each stream and concatenating to a 2176-d representation. With an ASFormer head this matches DiffAct on segmentation (F1@50 68.2) while, crucially, being the *only* configuration to produce non-zero fault detection (Sec. 5).

v4 — Fusion + DiffAct (best). The two wins are *orthogonal*: fusion improves the *features*, DiffAct improves the *head*. Combining them—feeding the 2176-d fusion features to DiffAct—gives our best model, **Acc 84.21 / Edit 87.19 / F1@50 79.50** in the updated project results (Fig. 1).

Table 1: Updated IndustReal step-segmentation scorecard supplied with the project results. Higher is better. The SAM3 row reports the enriched configuration separately from the best v4 model.

Configuration	Acc	Edit	F1@50
v1 Huge-K710 + ASFormer (baseline)	73.3	68.8	56.3
v1 + Viterbi	73.4	74.2	62.2
v2 SSv2-giant + ASFormer + Viterbi	74.0	77.3	66.5
v2 SSv2-giant + DiffAct	74.4	79.6	68.9
Fusion (giant+IV2-B14) + ASFormer + Viterbi	75.7	77.6	68.2
v4 Fusion + DiffAct (best)	84.21	87.19	79.50
v4 + SAM3 enrichment	74.9	79.5	70.1

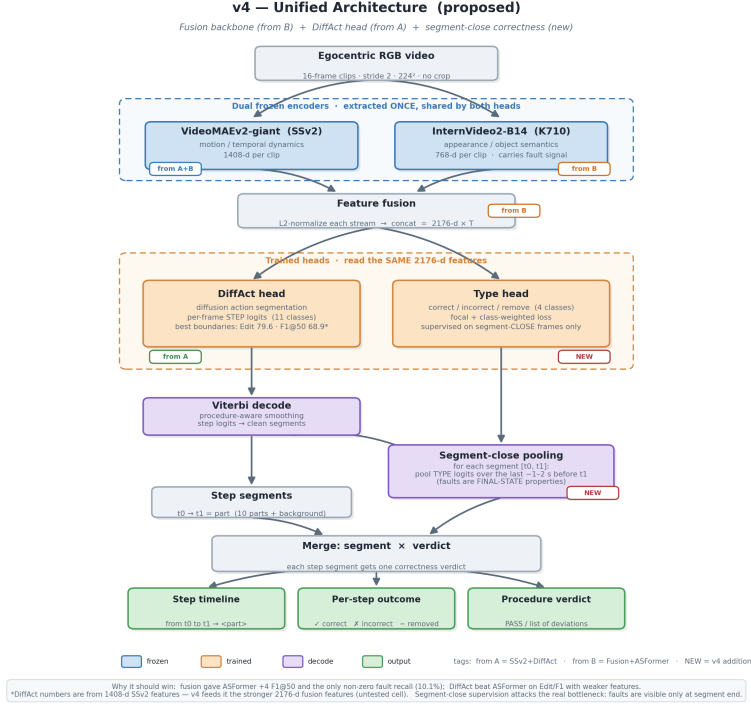


Figure 1: The v4 unified architecture: dual frozen encoders (VideoMAEv2-giant + InternVideo2-B14) → 2176-d fusion features → DiffAct step head; a parallel type head predicts correctness. Provenance tags mark which component each part inherits.

Real-time streaming (v3). For online use we build a fully causal variant: a distilled ViT-B backbone over a 16-frame window ending at t , MiniROAD-style causal GRU heads [8], and a fixed-lag online Viterbi that commits frame $t - L$. At $L=16$ (≈ 2.7 s latency) it reaches edit 55.3 / F1@50 37.4. A TeSTra head [9] under-performed the GRU (24.8 F1@50), consistent with transformers being data-hungry on our 36-recording training set.

Cross-dataset (MECCANO). Applying the same fusion+ASFormer recipe unchanged to MECCANO [2] (20 recordings, 18 step classes) gives F1@50 38.0, confirming the framework generalizes.

5 The correctness sub-problem

Correctness (*was the step done right?*) is markedly harder than segmentation. The type head’s incorrect-install recall is 0% for any VideoMAE-only backbone and rises to 10.1% only with the InternVideo2 fusion—the first non-zero fault detection (Table 2). A larger appearance encoder (InternVideo2-L14) *regressed* to 0%, and on MECCANO recall is also 0%. We conclude correctness is **data-limited, not method-limited**: with only ~ 29 labeled incorrect events, encoder capacity is not the lever—more and sharper error labels are.

Table 2: Correctness (incorrect-install recall) and real-time streaming summary.

<i>Correctness — type head, IndustReal</i>		
Configuration	Incorrect recall	Remove recall
Any VideoMAE-only + ASFormer	0.0%	~68%
Fusion giant + InternVideo2-B14	10.1%	69.4%
Fusion giant + InternVideo2-L14	0.0%	—
<i>Real-time streaming — IndustReal, causal</i>		
Configuration	F1@50	Latency
v3 causal ViT-B + GRU (L=16)	37.4	2.7 s
v3 TeSTra head (L=16)	24.8	2.7 s

5.1 Can a promptable foundation model judge correctness?

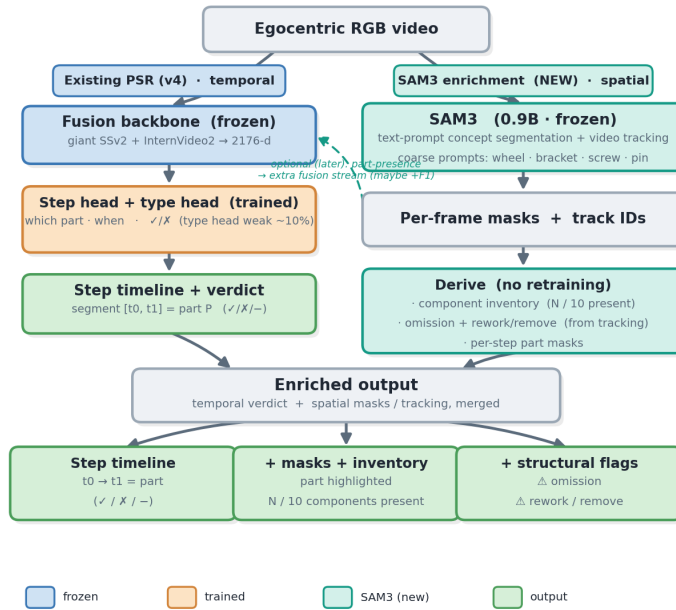
We tested two recent models on our hardware (AMD MI210 / ROCm). **LocateAnything** [13] (a 3B grounding VLM) and **SAM3** [12] (a 0.9B promptable segmenter) both reliably *localize* the STEMFIE parts for coarse categories (wheel, screw, bracket). However, on a part-matched set of ~57 correct-vs-incorrect installs, neither could *judge* correctness: LocateAnything echoed the prompt as a referring expression and returned boxes regardless of the true label (100% recall / 51% precision = chance), and SAM3 segments a part whether or not it is faulty. Both are *locators*, *not judges*; correctness requires a downstream comparison against a reference, which no perception model supplies.

6 SAM3 enrichment layer

Given the above, we use SAM3 not for correctness but to **enrich the output** at inference (Fig. 2). SAM3 produces pixel-accurate instance masks (far sharper than the boxes of the VLM) and supports video tracking. We wire it on top of the frozen v4 pipeline: for each committed step segment we run SAM3 with the coarse category of the step’s part and overlay the resulting masks, and we aggregate a per-recording **component inventory**. The result is the same v4 timeline augmented with (i) per-step part masks (the “where”) and (ii) a component inventory (“what is present”), with *no retraining and identical accuracy* (Fig. 3). SAM3 masks and video tracking further enable, as future work, *omission* and *rework* detection (comparing the inventory to an expected bill-of-materials, and tracking part identities over time)—signals that are not data-limited because they do not depend on the scarce incorrect labels.

PSR pipeline + SAM3 enrichment

existing PSR gives the temporal timeline · SAM3 adds spatial masks + tracking at inference



What SAM3 adds

Perception, not a judge

Masks + tracking — the WHERE. It does not classify correct/incorrect (proven: seg ≠ judge).

New info, no retraining

At inference: per-step masks, a live component inventory, and omission / rework flags.

Not data-limited

Omission & rework/remove come from tracking, not the scarce 'incorrect' labels.

Coarse only

wheel/bracket/screw segment well; fine part identity stays with the PSR step head.

Caveats

License gated (Meta terms); ROCm unproven; masks don't move F1 by themselves.

Do first: enrichment (#1-#2) — low-risk, no retraining, visibly richer output. Defer the feature-stream / any correctness use.

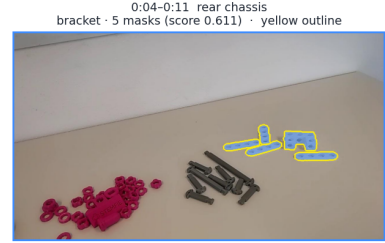
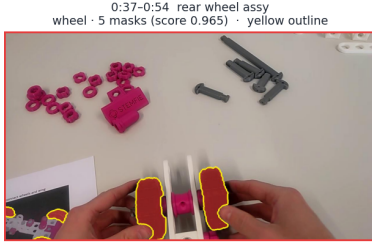
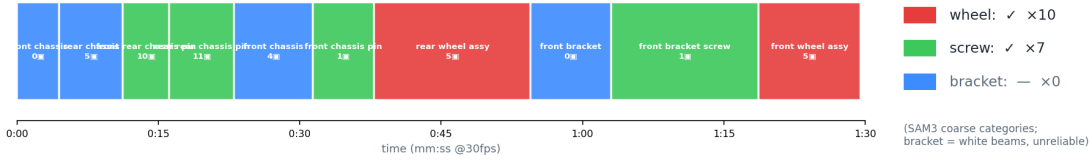
SAM3 is an inference-time enrichment lane — it makes the output richer and adds omission/rework detection, without touching (or fixing) the weak type head.

Figure 2: The v4 pipeline (temporal: which/when) with an added SAM3 enrichment lane (spatial: masks + inventory), converging into an enriched output. SAM3 is an inference-time add-on and does not alter the model or its accuracy.

Enriched output — 03_assy_0_1

PSR v4 timeline (which / when) + SAM3 masks (where) + component inventory

Step timeline (segment colored by SAM3 category · ■ = mask instances)



SAM3 supplies the WHERE (pixel masks) + a component inventory; the PSR head supplies WHICH/WHEN. No correctness claim — this is inference-time enrichment, not a fix to the type head.

Figure 3: Enriched output for one recording: the v4 step timeline (colored by SAM3 category), a component inventory, and per-step SAM3 mask overlays (bright outlines). SAM3 supplies the *where*; the v4 head supplies the *which/when*.

7 Results and Discussion

Table 1 records the updated IndustReal comparison. Three findings recur. (1) **Backbone > head for segmentation:** the SSv2-giant features help every head, and fusion helps further. (2) **The two wins are orthogonal:** better features and a better temporal head combine in v4. (3) **Correctness is data-limited:** neither larger encoders nor foundation-model VLMs move it; the lever is more error data and sharper labels. SAM3 enrichment remains an output-enrichment path rather than a substitute for a correctness classifier.

The engine-model dataset provides a separate, deployment-relevant benchmark. These results are reported for the engine-model corpus collected specifically for Assembly Copilot and should not be merged numerically with the IndustReal table: the datasets, splits, and evaluation stages are different. The offline v4 model is the strongest recognizer, while the online results quantify the cost and improvement of causal inference.

Table 3: Assembly Copilot results on the collected engine-model dataset. The online rows are the direct comparison between the initial causal baseline and the final online V1 model.

Configuration	Acc	Edit	F1@50
v4 Fusion + DiffAct (offline)	96.21	97.33	97.51
Baseline (online)	92.96	75.65	81.36
Final Online V1	94.12	84.45	89.50

8 From Research Benchmark to Assembly Copilot

The early experiments above established which representation and segmentation choices were promising on public benchmarks. The next phase changed the question from *which model scores best offline?* to *can a technician receive a stable, timely step announcement while the assembly is still happening?* This required new data, different temporal constraints, explicit stress testing, and an end-to-end demonstration path. The progression is summarized in Fig. 4.

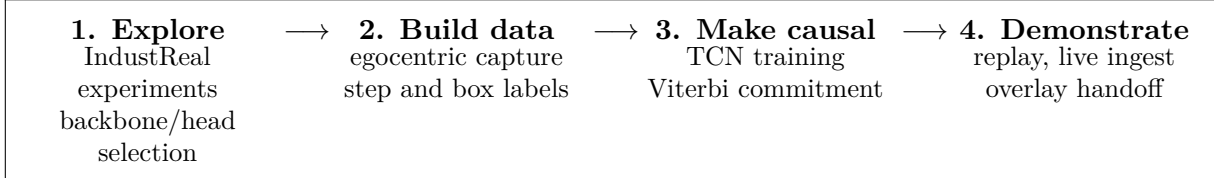


Figure 4: Development flow from the initial public-dataset research study to the tested live demonstrator. Each stage produced an artifact used by the next stage: measured experiments informed the causal design; recording and labeling tools produced the in-house corpus; the trained checkpoint powered replay and live inference.

8.1 IndustReal as the design filter

IndustReal was used as a controlled research environment rather than as the final production corpus. The public dataset made it possible to compare a VideoMAEv2-ASFormer baseline, Viterbi decoding, stronger Something-Something-v2 features, DiffAct, and two-stream fusion under one evaluation protocol. The resulting progression showed that the backbone and the temporal head contributed different gains: the dual-stream VideoMAEv2-InternVideo2 representation supplied complementary motion and appearance information, while DiffAct produced sharper offline boundaries. The best offline configuration, v4, reached $F1@50 = 79.50$ in the updated project results; the separately reported SAM3-enriched configuration reaches $F1@50 = 70.1$.

That result did not directly define the live model. DiffAct performs iterative denoising over a sequence and is well suited to an already-recorded video, where future frames are available. A live assistant cannot wait for the rest of the recording or revise an announcement using information from the future. The project therefore retained the strongest useful feature recipe—frozen VideoMAEv2 giant SSv2 features fused with frozen InternVideo2 features—but replaced the non-causal segmentation head with a causal temporal convolutional network (TCN). This distinction is important: the IndustReal numbers describe the research progression, whereas the metrics in Sec. 8.6 describe the later in-house streaming system on a different task corpus.

8.2 Building the in-house corpus

The team built a complete data path instead of relying on a collection of manually assembled files. The `assembly_video_recorder` application turns an iPhone into a wireless egocentric camera. A host computer serves a local HTTPS page; the phone opens the page in Safari, sends a WebRTC preview, and records the full-quality video back to the host. The workflow supports camera and lens selection, torch control, snapshots, wake-lock behavior, and several recording qualities. The result is a local MP4 that preserves the viewpoint needed by the model without requiring a phone application or cloud upload [16].

The companion `assembly_video_labeler` application provides a browser-based annotation workflow. An annotator scrubs the recording and marks the frame at which each procedure step

is complete using number-key shortcuts. The application supports resumable sessions, video proxies, and multi-user claims so that two people do not silently edit the same recording. It exports three complementary representations: `PSR_labels.csv` for completion events, `segments.csv` for temporal segments, and `labels.json` for application-friendly metadata. These sparse completion events are expanded into the dense time grid consumed by the temporal model. A separate `box_part_labeler` can label visible component boxes on selected still frames and export COCO, YOLO, or native JSON; this supports the optional detector and spatial enrichment path, but is not required for step recognition.

Table 4: Main in-house data assets used by the final Assembly Copilot.

Asset	Recorded or derived content
Curated recordings	40 egocentric turbofan recordings
Train/test split	30 train, 10 held-out test recordings
Task coverage	22 assembly and 18 disassembly recordings
Operators	Three operators; the split is not subject-disjoint
Procedure vocabulary	10 modeled parts plus background
Source video	1920×1080 at 30 fps
Dense training labels	386,529 frame/time labels in the corpus report
Stress library	6 additional reorder, reverse, partial, and wrong-step variants

8.3 Assembly demonstrator and label vocabulary

The in-house data was collected around a physical turbofan-style engine model. The model provides a consistent assembly target, exposes the parts in a meaningful order, and gives the egocentric camera a changing object state to observe. The engine image in Fig. 5 is the visual reference used in the demonstrator materials; the recorded videos show operators handling the same class of model during assembly and disassembly.



Figure 5: Turboprop-style engine model used as the visual reference for the Assembly Copilot demonstrator and dataset collection.

The annotation vocabulary follows the procedure rather than individual pixels. An annotator marks the moment at which a part has been installed or removed; the labeling service converts that event into an IndustReal-compatible step label and a temporal segment. The ten modeled assembly steps are listed in Table 5. The corresponding remove IDs are retained because the corpus includes both assembly and disassembly recordings.

Table 5: Assembly Copilot step labels and their IndustReal-compatible identifiers. The key in the labeling app is shown in the first column; key 0 denotes the tenth step.

Key	Step label	Install ID	Remove ID
1	Main Drive Planet Gear	3	5
2	Pop Control Ring Gear	6	8
3	Pop Control Sun Gear	9	11
4	Compressor Casing	12	14
5	Exhaust	15	17
6	Exhaust Casing	18	20
7	Cowling Bracket	21	23
8	Frame Subassembly	24	26
9	Propeller Cone plate	27	29
0	Propeller Cone Tip	30	32

The labeling application is intentionally lightweight: it runs as a local FastAPI web service, streams the recording in the browser, supports frame-by-frame scrubbing, and lets the annotator press the step key when the completion event occurs. It also supports assembly/disassembly

direction, resumable labels, video claims for multi-user work, and export to `PSR_labels.csv`, `segments.csv`, and `labels.json`. Figure 6 shows the interface with a real engine-assembly recording loaded, the ten labels visible, and four example completion marks placed on the timeline.

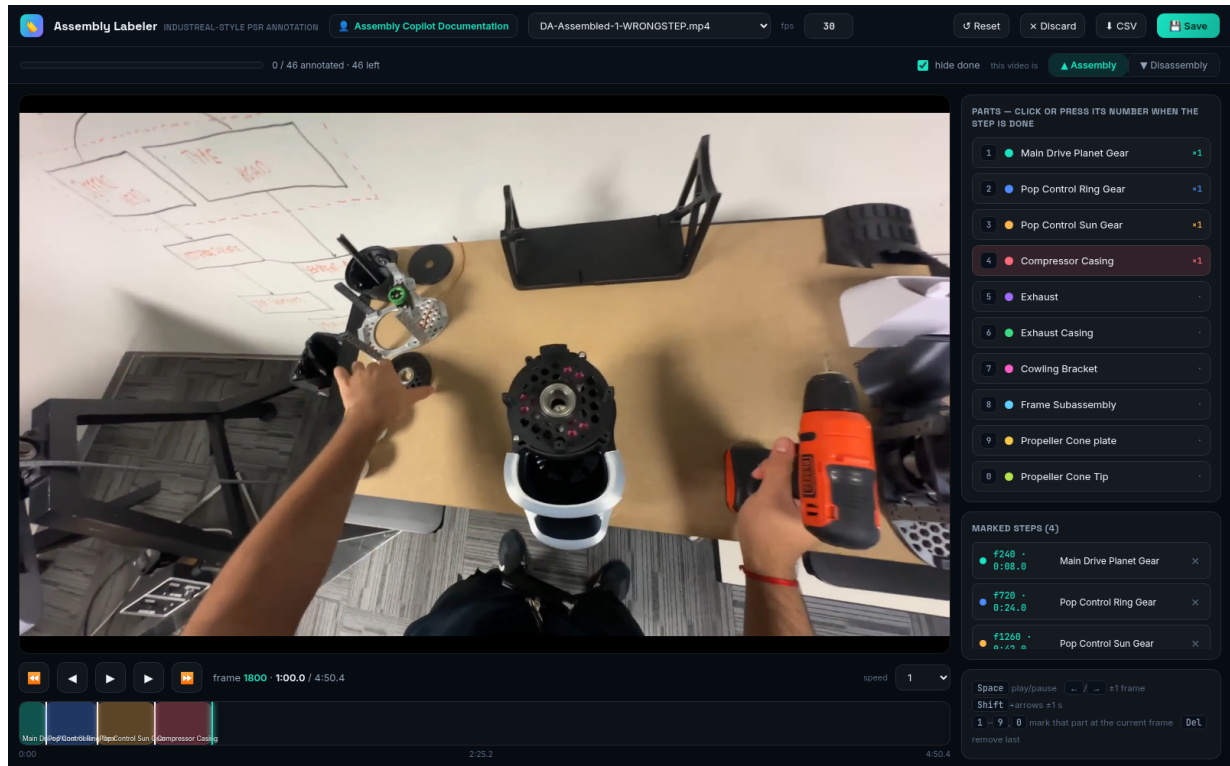


Figure 6: Assembly Video Labeler with an engine-assembly recording loaded. The annotator can inspect the video, select the assembly direction, choose one of the ten step labels, and mark the frame at which that step completes.

The stress variants deliberately test behavior that is easy to miss in a random held-out split: steps can arrive in an unexpected order, be omitted, be reversed, stop part-way through, or be replaced by a wrong-step event. The resulting library contains 46 playable assets (the 40 curated recordings plus six stress variants) and gives the decoder a concrete test of order awareness rather than only appearance matching. Because three operators occur in both sides of the train/test split, the reported test performance should be read as an engineering benchmark for this corpus, not as a subject-independent generalization claim.

8.4 The final causal streaming architecture

The deployed model processes the stream in short overlapping windows. Source video is sampled at 10 fps. Every third source frame is retained, and 16 retained frames form a 1.6s clip. A new clip is evaluated every 0.2s, so the system updates frequently while each feature vector still contains enough motion context. VideoMAEv2 giant SSv2 produces a 1408-dimensional motion-oriented feature and InternVideo2-B14 produces a 768-dimensional appearance/semantic feature. Each stream is ℓ_2 -normalized and concatenated into a 2176-dimensional vector. Both encoders remain frozen; only the temporal head and decoding logic are trained for the assembly vocabulary.

The head is a nine-block residual causal TCN with 128 channels and dilated temporal convolutions. It has approximately 875 K trainable parameters and a receptive field of about 205s.

Causality is verified in code by checking that perturbing a future input cannot change an earlier output. The head predicts 11 classes: the ten modeled parts and background. It does not announce every raw arg-max change. Instead, the log-probability sequence is passed to an online fixed-lag Viterbi decoder. The transition matrix is estimated from training annotations with Laplace smoothing, a self-transition bias favors stable steps, and a lag of ten model ticks corresponds to roughly 2 s.

This decoder is the mechanism that makes *a new step* operationally well defined. The model produces a new prediction every 0.2 s, but the application announces a step only when the Viterbi path has committed a different class after the lag. Short-lived changes are therefore treated as uncertainty or flicker; a persistent change in the committed path becomes the next step event. This is also why the same decoder is used in replay and live serving: the offline result is not allowed to use more temporal information than the live result.

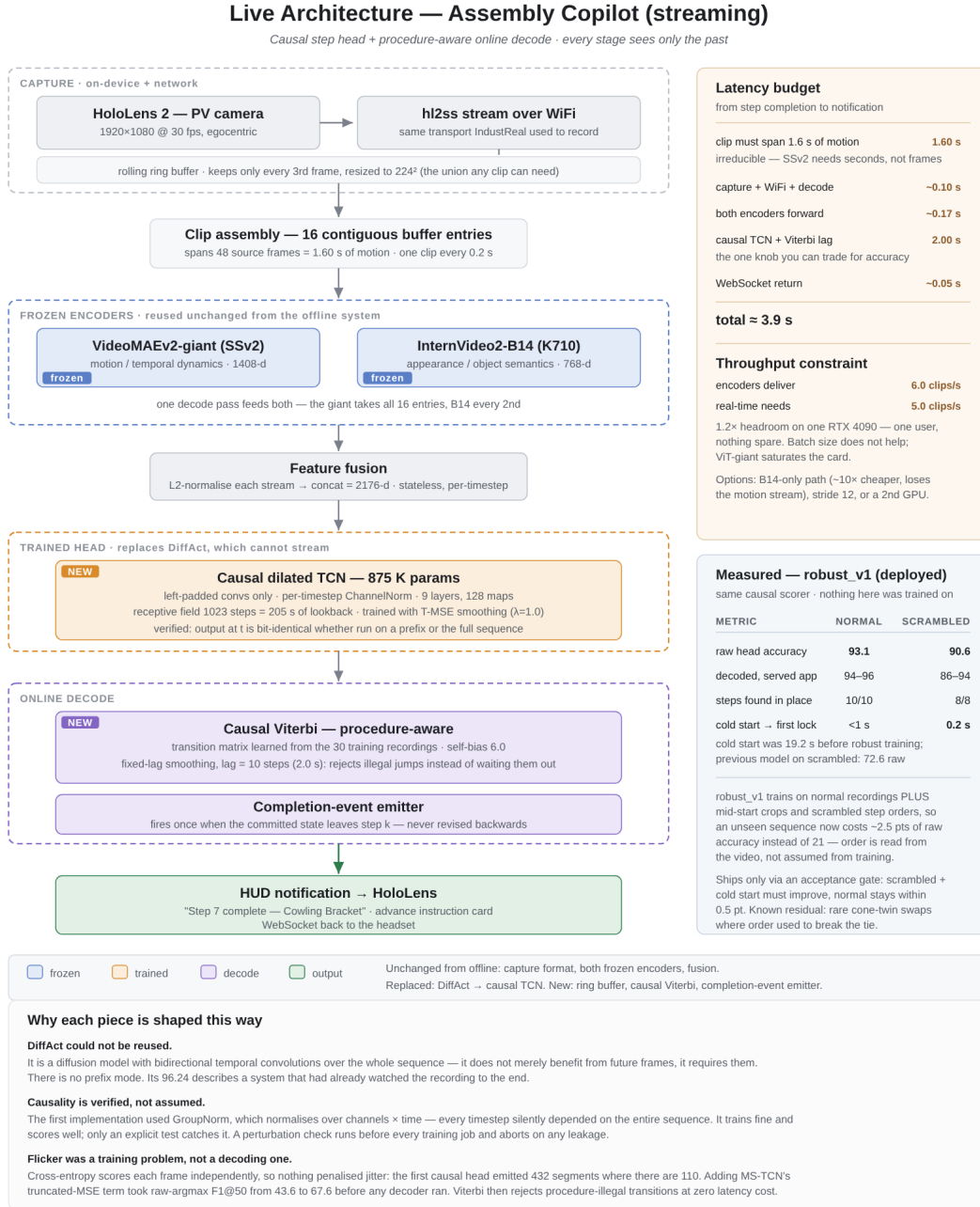


Figure 7: Final streaming architecture from the active `assembly_copilot` implementation. Frames are buffered into 1.6s clips, converted by frozen dual encoders into a 2176-dimensional representation, classified by a causal TCN, and committed by fixed-lag online Viterbi decoding. The figure also reports the measured latency and robustness results of the deployed checkpoint.

8.5 Training, stress testing, and model iteration

The training loop was designed around the failure modes that matter for a live assistant. It uses class-weighted cross-entropy, a smoothness regularizer based on truncated mean-squared error, and on-the-fly temporal augmentation. Augmentations include crops that force the model to start in the middle of a procedure and reordered sequences that reduce dependence on the normal step

order. The acceptance process then tests the trained checkpoint on real stress videos rather than relying only on augmented training batches.

Table 6: Model iteration and acceptance-gate results for the in-house streaming system. Normal and scrambled scores are held-out decoded agreement; cold start is the time until the first usable committed step.

Candidate	Recipe	Normal	Scrambled	Cold start	Decision
Baseline	TMSE 1.0, dropout 0.25	93.45%	72.58%	19.2 s	Reference
Fine-tune A	Fine-tuned candidate	92.39%	91.4%	–	Rejected: normal floor
Fine-tune A2	Second fine-tuned candidate	92.95%	88.84%	–	Passed, but not selected
robust_v1	Scratch training, epoch-25 winner	93.06%	90.57%	0.2 s	Deployed

The table captures an important engineering result: the best candidate was not the one with the highest single stress score. Fine-tune A improved scrambled-order behavior but reduced normal performance below the acceptance floor. Fine-tune A2 passed the floor, while the scratch-trained **robust_v1** checkpoint gave the best balance and dramatically reduced cold-start delay. The deployed checkpoint is stored as `checkpoints/deployed.pt` and points to `robust_v1.pt`.

The acceptance gate used three explicit conditions: normal performance at least 92.95%, scrambled-order performance better than the baseline, and cold start shorter than 19.2 s. This separates normal recognition quality, procedural order robustness, and responsiveness. It also prevents a model from being accepted because it only memorizes the most common assembly order.

8.6 From replay to live inference

The software was brought up in stages so that model errors could be separated from serving errors. First, cached features and labels were used to make training and evaluation repeatable. The replay server then supported two paths: fast playback from precomputed features and a slower full-encoding path that runs the encoders on the recording. It exposes a browser timeline with predicted steps, ground truth where available, saliency information, and optional detector output. This made it possible to inspect a complete prediction before introducing network latency.

The final serving path uses the same feature encoders, TCN, and online Viterbi decoder. A phone or driver supplies a 10-fps live stream, the server maintains the 16-frame buffer, and the committed state is exposed through the live API. The deployed implementation runs close to real time at approximately $1.14\times$ playback speed. The measured end-to-end announcement delay is approximately 2.84 s, consistent with the 1.6 s clip context and roughly 2 s of fixed-lag stabilization, with overlap and centering reducing the simple sum.

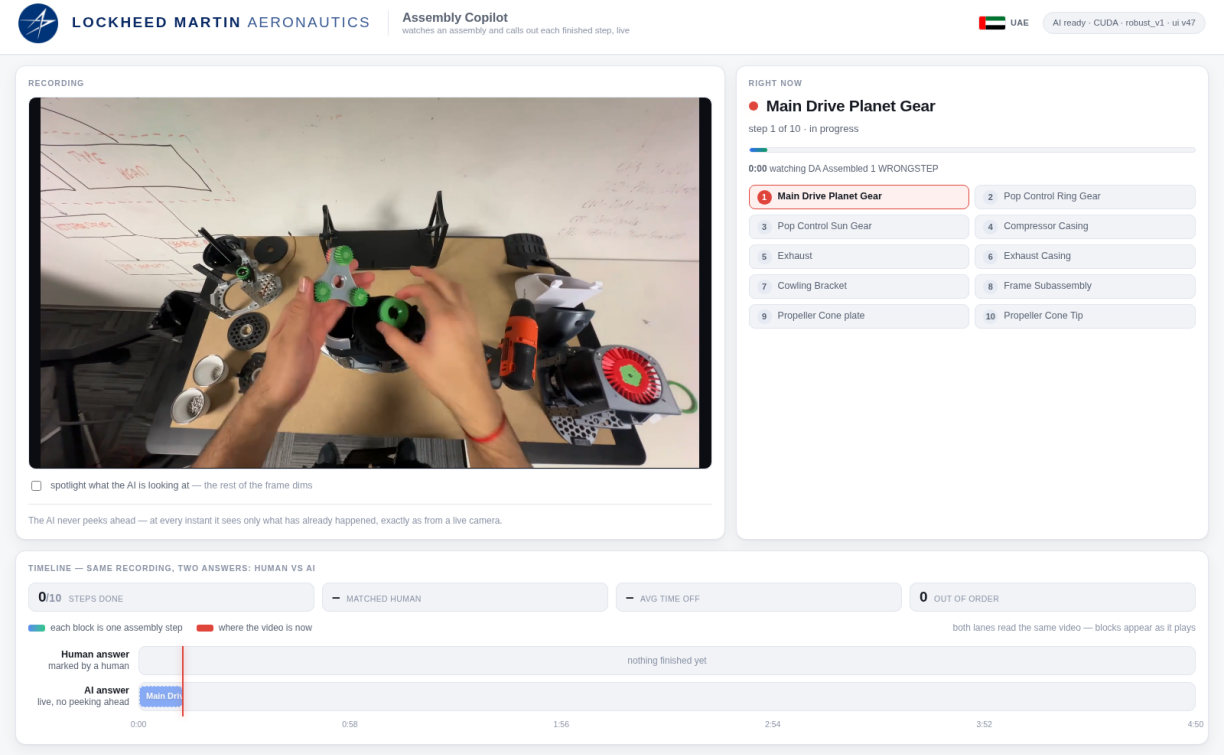


Figure 8: Replay inference interface with an engine-assembly recording loaded. The browser presents the live recording, current predicted step, ten-step procedure state, progress, and the human-versus-AI timeline. The same causal head and fixed-lag decoder are used here as in the live serving path.

Table 7: Served replay verification using the deployed `robust_v1` checkpoint. *Steps in place* counts expected procedure steps recovered by the decoder; these examples are complementary to the aggregate acceptance-gate scores in Table 6.

Recording	Agreement	Steps in place
Nahom-Assembled-5	93.9%	10/10
DA-disAssembled-7	96.1%	10/10
Ketan-Assembled-9	94.0%	10/10
Ketan-Assembled-7-SCRAMBLED	93.9%	10/10
DA-Assembled-1-SCRAMBLED	91.5%	8/8
Nahom-Assembled-7-PARTIAL	92.5%	7/7
DA-Assembled-2-REVERSED	89.8%	10/10
Nahom-disAssembled-13-SCRAMBLED	85.5%	8/9

8.7 Demo and HoloLens handoff

The live demo is intentionally split into a model service and a display client. The `holo-lens-api-app` accepts live frames, runs inference, and publishes compact state such as the current step, expected next step, confidence, progress, warning status, latency, and frame rate. A small overlay publisher maps the ten model classes to the visible thirteen-part assembly catalogue. Three model classes span two visible parts, so this mapping is kept in configuration rather than hidden in the network.

The display side is the separate Unity UWP HoloLens application used by the operations-helper

project [17]. The local overlay service receives state changes through `/step`; the Unity client polls `/state` and updates the display. The application contains a scripted demo mode for rehearsals and a live mode for model-driven updates. The screenshot in Fig. 9 shows the resulting technician-facing panel with the Lockheed Martin-branded header, current part, reference image, progress, and telemetry.

The current HoloLens presentation is a movable, resizable 2-D screen-space window rather than a spatially anchored 3-D model overlay. The Unity project uses Microsoft Mixed Reality Toolkit and OpenXR, but the present integration does not use Vuforia image targets, model targets, or a tracked physical coordinate system. This boundary is useful for interpreting the demonstration: the model recognizes the procedure and sends state; Unity renders guidance. Spatial anchoring can be added later without changing the temporal recognition core.



Figure 9: Technician-facing HoloLens overlay used in the live demonstration. The current application renders a screen-space guidance window and receives compact state updates from the local overlay service; it is not the recognition model itself and does not currently perform spatial object registration.

8.8 Project structure, reproducibility, and dependencies

The implementation is organized around the experiment-to-demo flow. The active `assembly_copilot` repository contains the frozen encoders and weights in `copilot_model/`, training and acceptance scripts in `training/`, production annotations and cached features in `dataset/`, a browser replay server in `replay-demo/`, and the live API in `holo-lens-api-app/`. The GitLab project suite contains the recording, step-labeling, and optional component-box labeling applications [16]. The active repository’s `REPRODUCING.md` describes the same sequence used during development: prepare features and labels, create stress videos, train, run the acceptance gate, and finally exercise the served applications [15].

The final model assets and dataset metadata are kept with the active project, but the complete demonstration still has explicit external boundaries. The Python runtime requires the project environment and the video foundation model dependencies. The live HTTPS path uses certificates and source recordings from the local web-application workspace. The HoloLens display is a separate Unity 2020.3.28f1 project with MRTK/OpenXR packages and a local Python overlay

service. Recording these boundaries matters for reproducibility: a fresh checkout can reproduce the model and replay path, while the phone and HoloLens demonstration additionally require the local services and hardware/display project.

8.9 Limitations and lessons

Several limitations remain part of the result rather than being hidden by the demo:

- The in-house test split is not subject-disjoint, so the reported normal and stress scores are not a substitute for evaluation on unseen operators or a new assembly line.
- The current causal model recognizes the modeled procedure and its order; it is not yet a general wrong-installation judge. A wrong-step stress case tests a useful failure mode, but substantially more segment-close incorrect examples are needed for reliable correctness classification.
- A committed step is intentionally delayed by the fixed-lag decoder. This trades immediate but unstable predictions for announcements that a technician can trust; the measured delay is approximately 2.84s.
- The model vocabulary and visible overlay vocabulary are not identical: ten learned classes map to thirteen displayed parts. This is practical for the current turbofan demonstrator but should be replaced by an explicit procedure schema for broader deployments.
- The current HoloLens client is a display and interaction surface. It does not yet run the foundation encoders on-device or register instructions to physical parts. Future work includes subject-disjoint data, richer fault labels, edge optimization, spatial anchoring, and reference-based omission/rework checks.

9 Conclusion

This work has two connected outcomes. The first is a controlled research study on public procedure-recognition benchmarks: Viterbi decoding, stronger SSv2-giant features, DiffAct, and two-encoder fusion culminated in **F1@50 = 79.50**, while the separately reported SAM3-enriched configuration reached 70.1 and added spatial enrichment without pretending to solve correctness. The second is an end-to-end Assembly Copilot built from the lessons of that study. It includes its own egocentric recording and labeling workflow, a 40-recording turbofan corpus, a causal dual-encoder TCN, fixed-lag online decoding, explicit order and cold-start stress tests, replay verification, and a live phone-to-HoloLens demonstration path. The deployed **robust_v1** checkpoint achieves 93.06% on normal held-out data and 90.57% on scrambled-order stress data, with a measured 2.84s live delay. The clearest next step is to improve data quality and coverage: subject-disjoint evaluation, more precisely timed incorrect-installation examples, and reference-based checks for omission, rework, and physical correctness.

References

- [1] T. Schoonbeek *et al.*, “IndustReal: A Dataset for Procedure Step Recognition Handling Execution Errors in Egocentric Videos,” *WACV*, 2024.

- [2] F. Ragusa *et al.*, “The MECCANO Dataset: Understanding Human-Object Interactions from Egocentric Videos in an Industrial-like Domain,” *WACV*, 2021.
- [3] Y. A. Farha and J. Gall, “MS-TCN: Multi-Stage Temporal Convolutional Network for Action Segmentation,” *CVPR*, 2019.
- [4] F. Yi, H. Wen, and T. Jiang, “ASFormer: Transformer for Action Segmentation,” *BMVC*, 2021.
- [5] D. Liu *et al.*, “Diffusion Action Segmentation,” *ICCV*, 2023.
- [6] L. Wang *et al.*, “VideoMAE V2: Scaling Video Masked Autoencoders with Dual Masking,” *CVPR*, 2023.
- [7] Y. Wang *et al.*, “InternVideo2: Scaling Video Foundation Models for Multimodal Video Understanding,” *ECCV*, 2024.
- [8] S. An *et al.*, “MiniROAD: Minimal RNN Framework for Online Action Detection,” *ICCV*, 2023.
- [9] Y. Zhao and P. Krähenbühl, “Real-Time Online Video Detection with Temporal Smoothing Transformers,” *ECCV*, 2022.
- [10] A. Kirillov *et al.*, “Segment Anything,” *ICCV*, 2023.
- [11] N. Ravi *et al.*, “SAM 2: Segment Anything in Images and Videos,” *arXiv:2408.00714*, 2024.
- [12] Meta AI, “SAM 3: Segment Anything with Concepts,” Technical Report / [facebook/sam3](#), 2025.
- [13] NVIDIA, “LocateAnything-3B: A Grounding Vision-Language Model,” [nvidia/LocateAnything-3B](#), 2025.
- [14] Qwen Team, “Qwen2.5 Technical Report,” *arXiv:2412.15115*, 2024.
- [15] Assembly Copilot project, *README and REPRODUCING.md*, internal project repository, 2026.
- [16] Assembly Copilot project suite, *assembly_video_recorder*, *assembly_video_labeler*, and *box_part_labeler*, internal GitLab repository, 2026.
- [17] AI for Operations Helper, *HoloLens Operations Helper Overview and Model Integration Handoff*, internal Unity project documentation, 2026.